

重庆三峡科技大学数学与统计学院

课程名称： 大数据行业应用案例实训

题 目： 基于大模型机器人对话开发

指导老师： 杨铸

学 生： 何慧娴

学 号： 202306274204

专业班级： 2023 级大数据管理与应用 2 班

目录

第 1 章 引言	4
1.1 项目背景与意义	4
1.1.1 行业发展背景	4
1.1.2 项目建设意义	4
1.2 项目需求分析	4
1.2.1 功能性需求	4
1.2.2 非功能性需求	4
第 2 章 技术方案设计	5
2.1 系统架构设计	5
2.1.1 总体架构	5
2.1.2 技术栈选型	5
2.2 核心功能模块设计	5
2.2.1 大模型接入层	5
2.2.2 业务逻辑层	5
2.2.3 交互展示层	6
2.3 数据流程设计	6
第 3 章 系统实现过程	6
3.1 开发环境搭建	6
3.2 后端核心代码实现	6
3.2.1 路由分发配置 (urls.py)	6
3.2.2 大模型流式响应视图 (stream_generate)	7
3.2.3 大模型同步响应视图 (no_stream_generate)	8
3.2.4 人格化提示词工程	8
3.3 前端交互界面实现	8
3.3.1 流式对话页面 (stream_chat.html)	8
3.3.2 非流式对话页面 (no_stream_chat.html)	9
3.4 关键难点与解决方案	9
第 4 章 系统测试与运行结果	9
4.1 功能测试	9
4.1.1 流式输出测试	9

4.1.2 非流式输出测试	9
4.1.3 异常处理测试	9
4.2 运行效果展示	10
第 5 章 总结与展望	10
5.1 实训总结	10
5.2 心得体会	11
5.3 未来展望	11

第 1 章 引言

1.1 项目背景与意义

1.1.1 行业发展背景

随着大数据与人工智能技术的深度融合，大语言模型（Large Language Model, LLM）已成为推动各行业数字化转型的核心引擎。在客服、教育、医疗及娱乐等领域，传统的规则式聊天机器人因灵活性差、上下文理解能力弱，已难以满足用户日益增长的个性化需求。基于生成式 AI 的智能对话机器人，凭借其自然语言理解与生成能力，正逐步成为人机交互的新标准。特别是“流式输出”技术的应用，使得机器回复能够像真人一样逐字呈现，极大地降低了用户的等待焦虑，提升了交互体验。

1.1.2 项目建设意义

本项目旨在利用阿里云通义千问（Qwen）大模型能力，结合 Django Web 框架，开发一款具备“流式/非流式”双模式切换功能的智能对话机器人。

技术实践价值：深入掌握大模型 API 调用、Server-Sent Events (SSE) 流式传输协议以及前后端分离开发模式。

行业应用价值：探索大模型在垂直场景（如情感陪伴、智能客服）中的落地方式，通过定制化的“人格设定”（如“知心大姐姐”），验证提示词工程（Prompt Engineering）对提升用户粘性的作用。

教学示范价值：作为大数据实训案例，直观展示从数据输入、模型推理到结果可视化的全链路开发过程。

1.2 项目需求分析

1.2.1 功能性需求

多模式对话：系统需支持两种对话模式：

流式模式：服务端实时推送生成内容，前端逐字渲染，模拟真人打字效果。

非流式模式：服务端等待模型生成完毕后一次性返回，前端整体渲染。

人格化交互：机器人需具备特定人设（刘亦菲形象、知心大姐姐风格、擅长土味情话），确保回复语气亲切、幽默。

基础交互：提供简洁的 Web 界面，包含消息输入框、发送按钮及对话记录显示区。

接口服务：提供标准的 RESTful API 接口，支持获取用户信息及处理对话请求。

1.2.2 非功能性需求

实时性：流式模式下，首字延迟（Time to First Token）应控制在秒级以内。

稳定性：具备完善的异常处理机制，当网络波动或 API 超时，能友好提示用

户而非直接崩溃。

可扩展性: 代码结构模块化, 便于后续接入其他大模型或增加多轮对话记忆功能。

安全性: 对 API Key 进行妥善管理, 对用户输入进行基本校验。

第 2 章 技术方案设计

2.1 系统架构设计

2.1.1 总体架构

本系统采用经典的 B/S (Browser/Server) 三层架构:

表现层 (Client): 基于 HTML5 + CSS3 + JavaScript 构建, 负责用户界面展示及事件监听, 通过 AJAX/SSE 技术与后端通信。

业务逻辑层 (Server): 基于 Python Django 框架, 负责请求解析、参数校验、调用大模型 API、数据流封装及响应返回。

数据服务层 (Service): 对接阿里云 DashScope 开放平台, 调用通义千问 (Qwen-Max) 大模型进行文本生成推理。

2.1.2 技术栈选型

层级	技术组件	选型理由
后端框架	Python 3.x + Django 4.x	生态丰富, 开发效率高, 原生支持流式响应对象。
HTTP 客户端	Requests	简洁强大, 完美支持 stream=True 参数, 适合处理流式数据。
大模型服务	阿里云 DashScope (Qwen)	中文理解能力强, 支持流式输出, 性价比高。
前端技术	HTML5 + Native JS	无需额外框架, 利用 EventSource 或 fetch 即可实现轻量级流式接收。
通信协议	HTTP/HTTPS + SSE	SSE (Server-Sent Events) 是实现单向流式推送的标准协议, 比 WebSocket 更轻量。

2.2 核心功能模块设计

2.2.1 大模型接入层

该层负责与阿里云 API 交互。

输入构造: 将用户消息封装为 JSON 格式, 包含 model (模型版本)、messages (对话历史, 含 System Prompt)、temperature (创造性参数) 等。

流式控制: 通过设置 stream=True 参数, 指示 API 以 SSE 格式返回数据; 反之则返回完整 JSON。

数据解析: 解析 API 返回的 data: {...} 行, 提取 choices[0].delta.content 字段。

2.2.2 业务逻辑层

路由分发: 根据 URL 路径 (/stream_generate/ vs /no_stream_generate/) 将请求分发至不同视图函数。

流式生成器：在 `stream_generate` 视图中，定义一个生成器函数（Generator），使用 `yield` 关键字逐个产出数据块，配合 `StreamingHttpResponse` 实现真正的流式传输。

同步处理：在 `no_stream_generate` 视图中，阻塞等待 API 完整响应，解析后通过 `JsonResponse` 返回。

2.2.3 交互展示层

流式页面：监听服务器事件流，每接收到一个数据包，立即操作 DOM 节点追加文本，实现“打字机”效果。

非流式页面：发送请求后显示“加载中”状态，收到完整响应后一次性替换内容。

2.3 数据流程设计

用户发起：用户在网页输入框输入“你好”，点击发送。

请求传输：前端构建 GET 请求（携带 `message` 参数），发送至 Django 后端。

后端处理：

视图函数接收请求，校验参数。

组装 Prompt（加入“你是刘亦菲...”系统指令）。

调用阿里云 API。

模型推理：阿里云云端大模型进行推理计算。

响应回传：

流式：数据分块返回 -> Django 生成器逐块 `yield` -> 浏览器逐字渲染。

非流式：数据整包返回 -> Django 解析 JSON -> 浏览器整段渲染。

结果展示：用户在屏幕上看到助手的回复。

第 3 章 系统实现过程

3.1 开发环境搭建

操作系统：Windows 10

Python 环境：安装 Python 3.9+，创建虚拟环境。

依赖安装：

```
pip install django requests
```

项目初始化：

```
django-admin startproject django_LLM
```

```
python manage.py startapp chat
```

3.2 后端核心代码实现

3.2.1 路由分发配置 (urls.py)

在主路由 `django_LLM/urls.py` 中，将 `/chat/` 路径委托给 `chat` 应用处理，并设置根路径重定向，优化用户体验。

```
from django.contrib import admin
```

```
from django.urls import path, include
```

```
from django.shortcuts import redirect
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('chat/', include('chat.urls')), # 子路由入口
    path("", lambda request: redirect('/chat/')), # 根路径重定向
]
```

注: chat/urls.py 中需定义具体的 stream_generate 和 no_stream_generate 路径映射。

3.2.2 大模型流式响应视图 (stream_generate)

这是本项目的核心亮点。利用 Python 生成器和 Django 的 StreamingHttpResponse 实现低延迟传输。

```
def stream_generate(request):
    if request.method == 'GET':
        message = request.GET.get('message')
        if not message:
            return JsonResponse({'code': 400, 'msg': '消息不能为空! '})

    def generate_stream():
        # 构造阿里云 API 请求
        headers = {"Authorization": f'Bearer {API_KEY}', "Content-Type":
"application/json"}
        data = {
            "model": "qwen3-max-preview",
            "messages": [
                {"role": "system", "content": "你是人工智能中的刘亦菲、
是一个知心大姐姐、还会说土味情话! "},
                {"role": "user", "content": message}
            ],
            "stream": True, # 开启流式
            "temperature": 0.7
        }

        response = requests.post(API_URL, headers=headers, json=data,
stream=True)

        for line in response.iter_lines():
            if line:
                decoded_line = line.decode('utf-8')
                if decoded_line.startswith('data:'):
                    # 解析 SSE 数据块
                    json_str = decoded_line[5:].strip()
                    if json_str == '[DONE]':
                        yield f'data: {{"done": true}}\n\n'
                        break
            try:
```

```

        chunk = json.loads(json_str)
        content = chunk['choices'][0]['delta'].get('content',
")
        if content:
            # 逐字推送
            yield f"data: {{"content": "{content}", "done":
false}}\n\n'

    except Exception:
        continue

    response = StreamingHttpResponse(generate_stream(),
content_type='text/event-stream')
    response['Cache-Control'] = 'no-cache'
    response['X-Accel-Buffering'] = 'no' # 防止 Nginx 缓冲
    return response

```

3.2.3 大模型同步响应视图 (no_stream_generate)

实现传统的请求 - 响应模式，用于对比测试。

```

def no_stream_generate(request):
    if request.method == 'GET':
        message = request.GET.get('message')
        # ... (省略参数校验) ...

        # 调用 API，默认 stream=False
        response = requests.post(API_URL, headers=headers, json=data)
        result = response.json()

        content = result['choices'][0]['message']['content']
        return JsonResponse({'code': 200, 'content': content})

```

3.2.4 人格化提示词工程

在代码中硬编码了系统提示词（System Prompt）：

"你是人工智能中的刘亦菲、是一个知心大姐姐、还会说土味情话！"

这一设定使得模型在回答时会自动调整语气，例如用户问“今天天气不好”，它可能会回答：“哎呀，心情不要像天气一样阴沉沉的嘛，姐姐给你讲个笑话暖暖心？”

3.3 前端交互界面实现

3.3.1 流式对话页面 (stream_chat.html)

使用原生 JavaScript 的 fetch API 配合 ReadableStream 读取器，或者使用 EventSource（需后端适配）来接收数据。

核心逻辑：

监听表单提交。

发起请求获取 Response 对象。
锁定 `response.body` 获取读取器。
循环读取 `chunk`，解码文本，通过正则提取 `content` 字段。
实时将文字追加到 `<div id="response-area">` 中。

3.3.2 非流式对话页面 (`no_stream_chat.html`)

核心逻辑：
监听表单提交。
发起 `fetch` 请求。
等待 `response.json()` 解析完成。
直接将 `data.content` 赋值给显示区域。
移除“加载中”提示。

3.4 关键难点与解决方案

难点：Django 默认会对响应进行缓冲，导致流式效果不明显。
解决：在响应头中设置 `X-Accel-Buffering: no` 和 `Cache-Control: no-cache`，强制禁用中间件和代理服务器的缓冲。
难点：阿里云 API 返回的 SSE 数据格式解析复杂，包含 `data:` 前缀和 `[DONE]` 标记。
解决：编写专门的解析逻辑，先去除前缀，再判断结束标记，最后使用 `json.loads` 安全解析内容。
难点：特殊字符（如引号、换行符）导致前端 JSON 解析失败。
解决：在后端 `yield` 之前，对 `content` 内容进行转义处理，或使用更稳健的字符串拼接方式。

第 4 章 系统测试与运行结果

4.1 功能测试

4.1.1 流式输出测试

测试用例：输入“请给我讲一个关于星星的土味情话”。
预期结果：点击发送后，屏幕立即开始逐字显示回复，无长时间等待，文字呈现平滑的打字机效果。
实际结果：符合预期。首字延迟约 0.5 秒，随后连续输出，体验流畅。

4.1.2 非流式输出测试

测试用例：同上。
预期结果：点击发送后，界面显示“思考中...”，约 2-3 秒后一次性显示完整段落。
实际结果：符合预期。直观展示了两种模式在时间维度上的差异。

4.1.3 异常处理测试

测试用例：输入空消息发送。
预期结果：后端返回 400 错误码，前端弹出“消息不能为空”提示。

实际结果：系统健壮，未发生崩溃，提示信息准确。

4.2 运行效果展示



流式模式运行截图：展示了“刘亦菲”助手正在逐字输出情话的瞬间，光标在最后一个字后闪烁。

非流式模式运行截图：展示了完整的对话气泡，内容一次性呈现。

控制台日志：显示了后端接收到的请求参数及 API 调用的状态码（200 OK）。

第 5 章 总结与展望

5.1 实训总结

本次实训成功开发了一款基于大模型技术的智能对话机器人。项目完整实现了从后端 Django 服务搭建、阿里云大模型 API 集成、流式传输协议应用到前端交互界面开发的全流程。

主要成果包括：

构建了支持流式（Streaming）和非流式（Blocking）双模式的对话系统。

通过提示词工程成功赋予了机器人“刘亦菲/知心大姐姐”的鲜明人格特征。

解决了 Django 环境下流式响应的缓冲问题，实现了低延迟的实时交互。

5.2 心得体会

通过本次开发，我深刻体会到了大模型应用开发的魅力与挑战。

技术层面：以前对“流式传输”只停留在概念层面，通过这次手写生成器（Generator）和配置 HTTP 响应头，真正理解了数据是如何在网络字节流中“流动”的。

应用层面：认识到 Prompt（提示词）是大模型的灵魂。简单的几句人设描述，就能让冷冰冰的代码变得有温度，这为未来从事垂直领域的大模型应用开发提供了重要思路。

工程层面：良好的代码规范和模块化设计（如将流式和非流式逻辑分离）极大地提高了调试效率。

5.3 未来展望

虽然当前系统已具备核心功能，但仍有改进空间：

多轮对话记忆：目前每次请求都是独立的，未来可引入 Session 或数据库存储对话历史，实现上下文关联的多轮对话。

前端框架升级：将原生 JS 替换为 Vue.js 或 React，利用其响应式数据绑定特性，使界面交互更加炫酷。

多模态扩展：接入大模型的语音识别（ASR）和语音合成（TTS）能力，实现真正的“语音对话”。

部署上线：使用 Docker 容器化技术，将项目部署至云服务器，供更多用户访问体验。

本次实训不仅提升了我的编程技能，更拓宽了我在大数据与人工智能交叉领域的视野，为未来的职业发展奠定了坚实基础。